

Trabajo 4 de Bases de Datos 2

Jeferson Garcia Angel
Sebastian Pabon Nuñez
Jhofred Jahat Camacho Gomez

1 Introducción

El objetivo de este trabajo es resolver y comparar varias formas de una misma consulta SQL: obtener todas las parejas de proveedores que venden exactamente el mismo conjunto de productos. La solución se desarrolla en Oracle y PostgreSQL, usando las mismas tablas y muestras de datos comparables.

El trabajo se hizo con Docker para que las pruebas puedan repetirse sin instalar manualmente los motores de base de datos en el sistema operativo.

2 Objetivos

- Crear las tablas **Prov** y **ProvxBProd** respetando la estructura del enunciado.
- Escribir cuatro consultas SQL distintas para resolver el problema.
- Crear un programa PL/SQL en Oracle que cargue datos aleatorios consistentes.
- Ejecutar pruebas con tamaños de muestra crecientes.
- Obtener Explain Plan y TKPROF en Oracle.
- Obtener métricas equivalentes en PostgreSQL mediante **EXPLAIN ANALYZE**.
- Comparar el comportamiento de las consultas en ambos motores.

3 Modelo De Datos

La tabla `Prov` almacena proveedores:

```
1 Prov(nit, nombre)
```

La tabla `ProvxBProd` almacena los productos que vende cada proveedor:

```
1 ProvxBProd(nit, codigoProducto)
```

Restricciones:

- `Prov.nit` es clave primaria.
- `Prov.nombre` es único.
- `(ProvxBProd.nit, ProvxBProd.codigoProducto)` es clave primaria compuesta.
- `ProvxBProd.nit` es clave foránea hacia `Prov`.

No se agregaron columnas adicionales porque el enunciado indica que no debe modificarse la estructura.

4 Metodología

Las pruebas se ejecutan con Docker Compose. Se levantan dos contenedores: Oracle Free y PostgreSQL.

Para cada muestra se realiza este proceso:

1. Se eliminan los datos anteriores.
2. Se cargan proveedores.
3. Se cargan relaciones proveedor-producto aleatorias, evitando duplicados.
4. Se ejecutan las cuatro consultas.
5. En Oracle se genera Explain Plan.
6. En Oracle se activa traza SQL y se procesa con TKPROF.
7. En PostgreSQL se ejecuta `EXPLAIN (ANALYZE, BUFFERS)`.
8. Se guardan los resultados en archivos de texto.

Los tamaños de muestra propuestos son:

Muestra	Proveedores	Productos posibles	Filas en ProvxProd
1	100	1.000	1.000
2	500	5.000	5.000
3	1.000	10.000	10.000
4	2.000	20.000	25.000
5	5.000	50.000	50.000

5 Generación De Datos

En Oracle se usa el procedimiento `cargar_datos_aleatorios`, escrito en PL/SQL. Primero borra los datos anteriores, luego inserta proveedores con `nit` consecutivo y nombre único. Después genera pares aleatorios (`nit`, `codigoProducto`). Si un par ya existe, se ignora y se intenta generar otro.

La clave foránea siempre es consistente porque los valores de `nit` se generan entre 1 y el número de proveedores creados. También pueden quedar proveedores sin productos. Esto es correcto, porque el enunciado contempla el caso de proveedores con conjunto vacío de productos. (Los primeros 20 proveedores se dejan sin productos para garantizar los casos de conjunto vacío).

En PostgreSQL se implementa una versión equivalente en PL/pgSQL.

6 Explicación De Las Cuatro Consultas

6.1 Consulta 1: doble NOT EXISTS

Esta consulta compara cada pareja de proveedores (`p1`, `p2`). La condición exige dos cosas: que no exista un producto vendido por `p1` que no venda `p2`, y viceversa. Esta consulta es distinta porque usa subconsultas correlacionadas y expresa la igualdad como ausencia de diferencias (división relacional).

6.2 Consulta 2: agrupación y firma del conjunto

Esta consulta crea una representación textual del conjunto de productos de cada proveedor (firma). En Oracle usa `LISTAGG`; en PostgreSQL usa `STRING_AGG`. Los productos se ordenan antes de concatenarlos.

Nota Técnica Importante: En Oracle, el operador `LISTAGG` tiene un límite de 4000 bytes (o 32767 con tipos extendidos). Si un proveedor vendiera una cantidad masiva de productos, esto generaría el error `ORA-01489: result of string concatenation is too long`. Para este experimento, el número de productos por proveedor no excede el límite, pero en producción esto

representa un riesgo de escalabilidad frente a enfoques puramente relacionales (como Q1 o Q3).

En PostgreSQL se reemplaza LISTAGG por STRING_AGG(x.codigoProducto::TEXT, ',' ORDER BY x.codigoProducto).

6.3 Consulta 3: conteos y productos comunes

Esta consulta calcula cuántos productos vende cada proveedor y cuántos tienen en común. Si tienen la misma cantidad total y los comunes coinciden con el total, venden el mismo conjunto. Es distinta porque se basa en agregaciones numéricas y no en operadores de conjuntos.

6.4 Consulta 4: operadores de conjuntos

En Oracle se usa MINUS; en PostgreSQL se usa EXCEPT. Revisa que las diferencias de conjuntos entre ambos proveedores sean vacías bidireccionalmente. En PostgreSQL se reemplaza MINUS por EXCEPT; el resto de la estructura es idéntica.

7 Resultados y Análisis

Todas las consultas retornaron 190 filas en todas las muestras, ya que se forzaron 20 proveedores sin productos, resultando en $20 \times 19/2 = 190$ parejas que comparten el conjunto vacío.

7.1 Tablas de Resultados

7.1.1 Tabla Resumen Oracle - Explain Plan

Muestra	Consulta	Cost	Elapsed	Observaciones
100 / 1.000	Q1 - NOT EXISTS	4472	0.03s	MERGE JOIN + FILTER, INDEX RANGE SCAN en PK_PROVXPROD
100 / 1.000	Q2 - LISTAGG	218	0.02s	TEMP TABLE TRANSFORMATION, HASH JOIN a tabla temporal
100 / 1.000	Q3 - conteos	26	0.03s	TEMP TABLE TRANSFORMATION, HASH GROUP BY, NESTED LOOPS
100 / 1.000	Q4 - MINUS	3356	0.02s	MERGE JOIN + MINUS con INDEX RANGE SCAN
500 / 5.000	Q1 - NOT EXISTS	4472	0.17s	Mismo plan, costo no refleja el crecimiento de datos
500 / 5.000	Q2 - LISTAGG	220	0.02s	Plan estable, costo sube ligeramente
500 / 5.000	Q3 - conteos	30	0.39s	Mismo plan, NESTED LOOPS sobre 124.750 pares
500 / 5.000	Q4 - MINUS	3356	0.25s	Mismo plan, MINUS se ejecuta por cada par
1.000 / 10.000	Q1 - NOT EXISTS	4472	0.72s	Plan idéntico, tiempo real crece
1.000 / 10.000	Q2 - LISTAGG	224	0.02s	Plan estable y eficiente
1.000 / 10.000	Q3 - conteos	41	2.22s	Plan hash, cambia a 2544319945 (full scan en Prov)
1.000 / 10.000	Q4 - MINUS	3356	1.03s	Mismo plan, 499.500 pares evaluados
2.000 / 25.000	Q1 - NOT EXISTS	4476	2.78s	Costo estimado apenas 4476, pero real 2.78s
2.000 / 25.000	Q2 - LISTAGG	234	0.07s	Plan estable, más rápido que Q1/Q3/Q4 por órdenes de magnitud
2.000 / 25.000	Q3 - conteos	63	10.32s	Plan adaptativo: PUSHED PREDICATE + INDEX RANGE SCAN (Plan Hash 3624992620); LIOs reducidos drásticamente en TKPROF
2.000 / 25.000	Q4 - MINUS	3360	4.08s	1.999.000 pares evaluados
5.000 / 50.000	Q1 - NOT EXISTS	165K	16.72s	Costo 165K, MERGE JOIN + INDEX RANGE SCAN, 12.5M filas
5.000 / 50.000	Q2 - LISTAGG	261	0.10s	Signe estable, TABLA TEMPORAL + HASH JOIN
5.000 / 50.000	Q3 - conteos	98	51.46s	Plan Hash 3380468438; NESTED LOOPS OUTER masivo (124M filas intermedias), 43.500 lecturas de disco
5.000 / 50.000	Q4 - MINUS	4486	25.29s	12.497.500 pares, 29.1M de LIOs, INDEX RANGE SCAN intensivo

7.1.2 Tabla Resumen Oracle - Métricas TKPROF

Se aplicó TKPROF a la traza SQL de cada consulta para obtener las tasas solicitadas por el enunciado. Las columnas de TKPROF se mapean así: f = *query* (consistent reads), g = *current* (current mode reads), h = *rows* (filas procesadas), i = filas retornadas (coincide con h para consultas SELECT), j = *fetches* (llamadas de fetch), k = *disk* (lecturas físicas de disco). Las métricas se corroboraron con la vista dinámica `v$sql` y los valores fueron consistentes.

Muestra	Consulta	LIOs $(f+g)/h$	Filas ret. $(i)/fetches (j)$	Disco $(k)/$ LIOs $(f+g)$
100 / 1.000	Q1 - NOT EXISTS	66.77	13.57	0.0000
100 / 1.000	Q2 - LISTAGG	0.10	13.57	0.0000
100 / 1.000	Q3 - conteos	76.48	13.57	0.0000
100 / 1.000	Q4 - MINUS	69.39	13.57	0.0000
500 / 5.000	Q1 - NOT EXISTS	1.550,49	13.57	0.0000
500 / 5.000	Q2 - LISTAGG	0.13	13.57	0.0000
500 / 5.000	Q3 - conteos	1.798,73	13.57	0.0000
500 / 5.000	Q4 - MINUS	1.554,50	13.57	0.0000
1.000 / 10.000	Q1 - NOT EXISTS	6.201,11	13.57	0.0000
1.000 / 10.000	Q2 - LISTAGG	0.23	13.57	0.0000
1.000 / 10.000	Q3 - conteos	7.717,08	13.57	0.0000
1.000 / 10.000	Q4 - MINUS	6.205,53	13.57	0.0000
2.000 / 25.000	Q1 - NOT EXISTS	25.251,64	13.57	0.0000
2.000 / 25.000	Q2 - LISTAGG	0.42	13.57	0.0000
2.000 / 25.000	Q3 - conteos	5,24	13.57	0.0000
2.000 / 25.000	Q4 - MINUS	25.282,46	13.57	0.0000
5.000 / 50.000	Q1 - NOT EXISTS	153.069,38	13.57	0.0000
5.000 / 50.000	Q2 - LISTAGG	0.74	13.57	0.0000
5.000 / 50.000	Q3 - conteos	34.195,11	13.57	0,0067
5.000 / 50.000	Q4 - MINUS	153.430,73	13.57	0.0000

Observaciones sobre la tabla TKPROF: La consulta Q2 mantiene los LIOs extremadamente bajos en todas las muestras (máximo 0,74 en la muestra más grande), confirmando su escalabilidad. La consulta Q3 muestra un comportamiento atípico en la muestra de 2.000 proveedores (5,24 LIOs/fila): el optimizador de Oracle seleccionó un plan adaptativo con *VIEW PUSHED PREDICATE* e *INDEX RANGE SCAN* (Plan Hash 3624992620), reduciendo drásticamente las lecturas lógicas respecto al plan anterior. En la muestra de 5.000 el optimizador volvió al plan masivo (Plan Hash 3380468438), registrando además 43.500 lecturas físicas de disco ($k/(f+g) = 0,0067$).

Muestra	Consulta	Execution Time	shared hit	shared read	Observaciones
100 / 1.000	Q1	24.02 ms	220	0	Hash Anti Join, Nested Loop, Seq Scan + Index Scan
100 / 1.000	Q2	1.04 ms	9	0	Hash Join con CTE, GroupAggregate
100 / 1.000	Q3	7.96 ms	216	0	Hash Join con CTE, HashAggregate, Nested Loop
100 / 1.000	Q4	56.40 ms	69.643	0	Nested Loop, SubPlan con HashSetOp Except
500 / 5.000	Q1	50.61 ms	2.522	0	Mismo plan: 500 x 250 filas en Nested Loop
500 / 5.000	Q2	3.01 ms	34	0	Plan eficiente, firma por STRING_AGG
500 / 5.000	Q3	157.76 ms	2.499	0	Hash Right Join produce 1.2M filas intermedias
500 / 5.000	Q4	1.397.49 ms	2.807.566	0	SubPlan se ejecuta 124.750 veces: 2.8M buffers
1.000 / 10.000	Q1	164.33 ms	200.728	0	Nested Loop produce 499.500 filas internas
1.000 / 10.000	Q2	5.44 ms	85	0	Plan estable, 10.020 filas agrupadas
1.000 / 10.000	Q3	635.50 ms	199.433	temp: 1511	Usa disco por primera vez (written: 1511 bloques)
1.000 / 10.000	Q4	6.613.12 ms	22.246.804	0	499.500 iteraciones del SubPlan: 22.2M buffers
2.000 / 25.000	Q1	1.380.45 ms	4.071.778	0	1.999.000 filas en Nested Loop: JIT habilitado
2.000 / 25.000	Q2	30.92 ms	436	0	Merge Join / Hash Join con STRING_AGG
2.000 / 25.000	Q3	3.684.11 ms	4.071.458	temp: 1803	32 batches hash, written: 1803 bloques
2.000 / 25.000	Q4	39.791.86 ms	99.268.558	0	1.999.000 iteraciones, Index Only Scan, 99.2M buffers
5.000 / 50.000	Q1	2.752.79 ms	133.063	0	Hash Anti Join + Nested Loop, 12.5M filas, JIT habilitado
5.000 / 50.000	Q2	34.99 ms	385	0	Merge Join, plan muy eficiente
5.000 / 50.000	Q3	13.178.98 ms	54.310	temp: 74682	Hash Join masivo, 257 batches, 145MB en disco temporal
5.000 / 50.000	Q4	54.898.74 ms	50.550.152	0	50.5M buffers, 12.5M iteraciones HashSetOp Except

7.1.3 Tabla Resumen PostgreSQL

7.2 Comparación de Rendimiento (Oracle vs PostgreSQL)

7.2.1 Tiempo de ejecución

La consulta más rápida fue Q2 (**LISTAGG** / **STRING_AGG**). Se mantuvo sumamente rápida (35 ms en Postgres y 0.10s en Oracle incluso con 5.000 proveedores) a pesar del crecimiento drástico de la muestra. La consulta más lenta en PostgreSQL fue Q4 (**EXCEPT**), alcanzando 54.90s de tiempo directo (y 80.71s en **EXPLAIN ANALYZE**) con 5.000 proveedores, mientras que en Oracle Q3 tomó 51.46s y Q4 tomó 25.29s para la misma muestra.

7.2.2 Lecturas lógicas y Planes de Ejecución

La diferencia de comportamiento en Q4 (donde PostgreSQL procesó hasta 99.2M de *shared hits* en la muestra de 2.000 proveedores vs Oracle con 29.1M *LIOs* en 5.000 proveedores) radica en la arquitectura del plan de ejecución:

- **PostgreSQL** anida el **EXCEPT** dentro de un *SubPlan* que se evalúa repetidamente para cada pareja posible de proveedores.
- **Oracle** utiliza un **INDEX RANGE SCAN** dentro de la operación **MINUS**. Esto ocurre porque Oracle aprovecha el índice compuesto (B-Tree) generado automáticamente en la Llave Primaria **ProvXProd(nit, codigoProducto)**. Al ser **nit** el primer atributo del índice, la base de datos puede resolver el **MINUS** eficientemente por índice sin recorrer bloques innecesarios.

8 Conclusiones

1. **La consulta más eficiente** fue Q2 (Firma con agregación de strings). Transformar un problema de conjuntos relacionales a un problema escalar evita el crecimiento cuadrático. Incluso con 5.000 proveedores se mantuvo por debajo de 0.10s en Oracle y 35 ms en PostgreSQL. Sin embargo, tiene la limitación de la longitud de bytes en Oracle.
2. **Operadores de conjuntos (MINUS / EXCEPT)**: Fueron los enfoques más intuitivos, pero los de peor rendimiento. En PostgreSQL con 5.000 proveedores alcanzó 54.90s (80.71s con EXPLAIN ANALYZE) debido a la ejecución anidada del EXCEPT en un SubPlan.
3. **El crecimiento cuadrático**: El problema base implica comparar combinaciones de N elementos ($N \times (N - 1)/2$). Q1, Q3 y Q4 escalan muy mal con grandes volúmenes. Con 5.000 proveedores se evalúan 12.5M de pares.
4. **Oracle vs PostgreSQL**: Oracle supo aprovechar sus índices por defecto de manera más eficiente en los operadores MINUS (25.29s vs 54.90s en la muestra más grande), mientras que PostgreSQL fue superior en la manipulación ultra rápida de STRING_AGG.

9 Herramientas de Rendimiento en PostgreSQL

El enunciado solicita investigar las herramientas equivalentes en PostgreSQL al Explain Plan y al TKPROF de Oracle. Se identificaron las siguientes:

- **EXPLAIN (ANALYZE, BUFFERS)**: Equivalente funcional al Explain Plan de Oracle más la ejecución real. La opción **ANALYZE** ejecuta la consulta y reporta tiempos reales; **BUFFERS** reporta los accesos a buffers (*shared hit*, *shared read*, *temp*), análogo a los LIOs y lecturas de disco de Oracle. Soporta múltiples formatos de salida (TEXT, JSON, YAML, XML).
- **pg_stat_statements**: Extensión que acumula estadísticas globales de cada consulta SQL ejecutada (tiempo total, llamadas, filas, buffers compartidos, buffers temporales, etc.). Equivalente funcional a la vista dinámica v\$sql de Oracle. Se habilita con `shared_preload_libraries = 'pg_stat_statements'` en `postgresql.conf`.

- **auto_explain:** Extensión que registra automáticamente los planes de ejecución de consultas lentas en el log del servidor. Cumple un rol similar a la combinación SQL_TRACE + TKPROF de Oracle: permite capturar planes de ejecución sin modificar el código de la aplicación. Se configura con `auto_explain.log_min_duration`.

En este trabajo se utilizó EXPLAIN (ANALYZE, BUFFERS) como herramienta principal de medición, ya que proporciona el plan de ejecución, tiempos reales y métricas de buffers en una sola instrucción, sin requerir configuración adicional del servidor.

10 Evidencias en PostgreSQL (Pantallazos)

A continuación se presentan los pantallazos solicitados en el enunciado que evidencian la ejecución de las pruebas y la obtención de métricas en PostgreSQL de forma local:

```
--- Métricas PostgreSQL (100 1000 1000) ---
Métricas disponibles en PostgreSQL
Para resultados comparables se usa EXPLAIN (ANALYZE, BUFFERS).
Buffers shared hit equivale aproximadamente a lecturas logicas desde cache.
Buffers shared read equivale a lecturas fisicas desde disco.
Actual rows muestra filas procesadas/retornadas segun cada nodo del plan.
  relname | heap_blks_read | heap_blks_hit | idx_blks_read | idx_blks_hit
-----+-----+-----+-----+-----
prov      |          0 |         2704 |          2 |         1808
provxprod |          0 |        88059 |          1 |        57122
(2 rows)
```

Figure 1: Métricas PostgreSQL - Muestra 100 proveedores

```
--- Métricas PostgreSQL (500 5000 5000) ---
Métricas disponibles en PostgreSQL
Para resultados comparables se usa EXPLAIN (ANALYZE, BUFFERS).
Buffers shared hit equivale aproximadamente a lecturas logicas desde cache.
Buffers shared read equivale a lecturas fisicas desde disco.
Actual rows muestra filas procesadas/retornadas segun cada nodo del plan.
  relname | heap_blks_read | heap_blks_hit | idx_blks_read | idx_blks_hit
-----+-----+-----+-----+-----
prov      |          0 |        21185 |          2 |        21021
provxprod |          0 |       4637884 |          1 |       1154733
(2 rows)
```

Figure 2: Métricas PostgreSQL - Muestra 500 proveedores

```

--- Métricas PostgreSQL (1000 10000 10000) ---
Métricas disponibles en PostgreSQL
Para resultados comparables se usa EXPLAIN (ANALYZE, BUFFERS).
Buffers shared hit equivale aproximadamente a lecturas logicas desde cache.
Buffers shared read equivale a lecturas fisicas desde disco.
Actual rows muestra filas procesadas/retornadas segun cada nodo del plan.
  relname | heap_blks_read | heap_blks_hit | idx_blks_read | idx_blks_hit
-----+-----+-----+-----+-----
prov      |          0 |    3127206 |          2 |    250373
provxprod |          0 |    37687165 |          1 |    5554749
(2 rows)

```

Figure 3: Métricas PostgreSQL - Muestra 1000 proveedores

```

--- Métricas PostgreSQL (2000 20000 25000) ---
Métricas disponibles en PostgreSQL
Para resultados comparables se usa EXPLAIN (ANALYZE, BUFFERS).
Buffers shared hit equivale aproximadamente a lecturas logicas desde cache.
Buffers shared read equivale a lecturas fisicas desde disco.
Actual rows muestra filas procesadas/retornadas segun cada nodo del plan.
  relname | heap_blks_read | heap_blks_hit | idx_blks_read | idx_blks_hit
-----+-----+-----+-----+-----
prov      |          0 |   28884893 |          2 |   550472
provxprod |          0 |  212103320 |          1 |  21988213
(2 rows)

```

Figure 4: Métricas PostgreSQL - Muestra 2000 proveedores

```

--- Métricas PostgreSQL (5000 50000 50000) ---
Métricas disponibles en PostgreSQL
Para resultados comparables se usa EXPLAIN (ANALYZE, BUFFERS).
Buffers shared hit equivale aproximadamente a lecturas logicas desde cache.
Buffers shared read equivale a lecturas fisicas desde disco.
Actual rows muestra filas procesadas/retornadas segun cada nodo del plan.
  relname | heap_blks_read | heap_blks_hit | idx_blks_read | idx_blks_hit
-----+-----+-----+-----+-----
prov      |          0 |   41986718 |          2 |   1124113
provxprod |          0 |  728135026 |          1 |  181601230
(2 rows)

```

Figure 5: Métricas PostgreSQL - Muestra 5000 proveedores

RESUMEN COMPARATIVO DE TIEMPOS DE EJECUCIÓN (NORMALIZADO A MS / SEG)		
Muestra: 100 prov / 1.000 rel (100prov_1000rel)		
Operación / Consulta	Oracle 23c (ms / s)	PostgreSQL 16 (ms / s)
Carga de Datos (Procedure)	60.00 ms	16.68 ms
Q1 (NOT EXISTS)	30.00 ms	24.02 ms
Q2 (STRING_AGG/LISTAGG)	20.00 ms	1.04 ms
Q3 (COUNTS)	30.00 ms	7.96 ms
Q4 (EXCEPT/MINUS)	20.00 ms	56.40 ms
Muestra: 500 prov / 5.000 rel (500prov_5000rel)		
Operación / Consulta	Oracle 23c (ms / s)	PostgreSQL 16 (ms / s)
Carga de Datos (Procedure)	340.00 ms	59.32 ms
Q1 (NOT EXISTS)	170.00 ms	50.61 ms
Q2 (STRING_AGG/LISTAGG)	20.00 ms	3.01 ms
Q3 (COUNTS)	390.00 ms	157.76 ms
Q4 (EXCEPT/MINUS)	250.00 ms	1,397.49 ms (1.40 s)
Muestra: 1.000 prov / 10.000 rel (1000prov_10000rel)		
Operación / Consulta	Oracle 23c (ms / s)	PostgreSQL 16 (ms / s)
Carga de Datos (Procedure)	1,200.00 ms (1.20 s)	117.23 ms
Q1 (NOT EXISTS)	720.00 ms	164.33 ms
Q2 (STRING_AGG/LISTAGG)	20.00 ms	5.44 ms
Q3 (COUNTS)	2,220.00 ms (2.22 s)	635.50 ms
Q4 (EXCEPT/MINUS)	1,030.00 ms (1.03 s)	6,613.12 ms (6.61 s)
Muestra: 2.000 prov / 25.000 rel (2000prov_25000rel)		
Operación / Consulta	Oracle 23c (ms / s)	PostgreSQL 16 (ms / s)
Carga de Datos (Procedure)	4,070.00 ms (4.07 s)	397.83 ms
Q1 (NOT EXISTS)	2,780.00 ms (2.78 s)	1,380.45 ms (1.38 s)
Q2 (STRING_AGG/LISTAGG)	70.00 ms	30.91 ms
Q3 (COUNTS)	10,320.00 ms (10.32 s)	3,684.11 ms (3.68 s)
Q4 (EXCEPT/MINUS)	4,080.00 ms (4.08 s)	39,791.86 ms (39.79 s)
Muestra: 5.000 prov / 50.000 rel (5000prov_50000rel)		
Operación / Consulta	Oracle 23c (ms / s)	PostgreSQL 16 (ms / s)
Carga de Datos (Procedure)	3,510.00 ms (3.51 s)	605.42 ms
Q1 (NOT EXISTS)	16,720.00 ms (16.72 s)	2,752.79 ms (2.75 s)
Q2 (STRING_AGG/LISTAGG)	100.00 ms	34.99 ms
Q3 (COUNTS)	51,460.00 ms (51.46 s)	13,178.98 ms (13.18 s)
Q4 (EXCEPT/MINUS)	25,290.00 ms (25.29 s)	54,898.74 ms (54.90 s)

Figure 6: Resumen Comparativo General